## TELL ME ABOUT YOURSELF ?

Good morning, and thank you for giving me this opportunity to introduce myself.

My name is Kailash Anshu Kumar Roy. I am currently pursuing my B.Tech in Artificial Intelligence and Data Science at Pace Institute of Technology and Sciences, Andhra Pradesh. I am currently in my fourth year, 7th semester, with a CGPA of 8.5.

Coming to my technical background, I have a strong foundation in Python, Java, DBMS, Data Structures and Algorithms, and Object-Oriented Programming. I enjoy working with computer science concepts and solving problems through programming.

I have also gained practical experience through internships. During my internship at IBM SkillBuild, I worked on machine learning-related tasks, including data preparation and working with real-world datasets. This experience helped me understand how theoretical concepts can be applied to practical problems. I have also completed internships at Brakes India and Surftech Engineering, where I gained exposure to professional work environments and learned the importance of teamwork and responsibility.

Apart from academics and internships, I have participated in hackathons, where I worked with teams to develop solutions within a limited time. I have also worked on a machine learning chatbot project, which gave me practical exposure to building AI-based applications.

I am also working as a Campus Ambassador at Aenexz, which has helped me improve my communication, leadership, coordination, and interpersonal skills.

Apart from academics, I enjoy playing cricket and volleyball. I have played both sports as part of teams, and these experiences have taught me the importance of teamwork, discipline, communication, taking instructions, and working toward a common goal.

Overall, I consider myself a continuous learner. I am always interested in learning new technologies and improving both my technical and communication skills.

At this stage of my career, I am looking for an opportunity where I can apply my technical knowledge, learn from experienced professionals, contribute to real-world projects, and grow as a software engineer.

That's a brief introduction about me. Thank you.

# What role can AI play in cyber security?

# Part 1: The simple answer (say this first, in 20 seconds)

"AI helps security teams find attacks faster. Companies get millions of logs and alerts every day, and humans cannot read them all. AI reads them quickly, finds anything unusual, and helps the team respond. But AI can also be attacked, so humans must always check the important decisions."

If the interviewer stops you here, this is already a good answer. If they ask for more, go to the next parts.

# Part 2: Where AI helps (explained simply)

**1. Finding threats (anomaly detection)**

- AI first learns what is normal. For example, Ravi logs in from Hyderabad at 10 AM every day.
- One day, "Ravi" logs in from another country at 3 AM. AI sees this is not normal and raises an alert.

**2. Catching malware**

- Old antivirus matched files against a list of known viruses. A brand-new virus was not on the list.
- AI watches behaviour. If a file starts locking many files quickly, AI says "this looks like ransomware" even if it has never seen it before.

**3. Stopping phishing emails**

- AI reads the email and checks the words, the links, and the sender.
- Words like "urgent, click now, your account will close" and a fake link make the email look risky, so AI moves it to spam.

**4. Spotting stolen accounts and insider threats**

- This is called user behaviour analytics. AI tracks what each user normally does.
- If an employee who normally downloads 50 MB suddenly downloads 10 GB, AI flags it as possible data theft.

**5. Helping the SOC team**

- A Security Operations Centre gets thousands of alerts, and many are false alarms.
- AI sorts them so analysts first look at the real and dangerous ones. AI tools can also explain a confusing log or write a short incident summary.

**6. Fraud detection**

- Banks use AI to check each card payment in a fraction of a second.
- If you normally buy groceries in Visakhapatnam and suddenly a big payment comes from another country, the bank blocks it.

**7. Fixing weak points first (vulnerability management)**

- A company may have thousands of software weaknesses. AI helps decide which few are most likely to be attacked, so the team fixes those first.

# Part 3: How does it work? (step by step)

1. **Collect data:** logs from firewalls, servers, laptops, emails, and cloud apps.
2. **Clean the data:** remove mistakes and repeated entries.
3. **Pick useful details (features):** for example, number of failed logins per hour, login time, and location.
4. **Train a model:**
    - If we have examples marked "attack" and "normal", we use **supervised learning**.
    - If we do not have labels, we use **unsupervised learning** to find what looks different from normal.
5. **Test the model:** check how many real attacks it catches and how many false alarms it gives.
6. **Use it live:** the model gives every event a risk score.
7. **Take action:** low risk is ignored, medium risk goes to an analyst, and high risk can be blocked automatically.
8. **Keep improving:** attackers change their methods, so we retrain the model regularly.

# Part 4: "How will YOU do it?" (use this example project)

Choose one project and explain it clearly. Here is a phishing email detector.

**Step 1: Get data.** I would collect many phishing emails and normal emails. Public sources such as PhishTank and open phishing datasets can help.

**Step 2: Prepare the data.** I would remove extra symbols and split the text into words. I would also create extra details like URL length, number of links, and age of the sender's domain.

**Step 3: Turn text into numbers.** A model cannot read words, so I would use TF-IDF or BERT embeddings.

**Step 4: Train the model.** I would start with Logistic Regression or Random Forest because they are simple and fast. Then I would try BERT to see if it gives better results.

**Step 5: Check the results.** I would not look only at accuracy. Phishing mails are rare, so a model can look 99 percent accurate by calling everything safe. I would check:

- **Recall:** how many real phishing emails we caught.
- **Precision:** how many of our alerts were really phishing.

**Step 6: Use it.** I would connect it to the email system. Clearly bad emails go to quarantine. Unsure ones go to a human analyst.

**Step 7: Monitor.** Attackers keep changing their wording, so I would collect new examples and retrain the model.

# Part 5: The risks (this makes your answer stronger)

"AI is useful for defenders, but it is also used by attackers."

- **Attackers use AI** to write very good phishing emails, clone voices, and make deepfake videos.
- **Attackers can trick AI models** by slightly changing the input. This is called an adversarial attack.
- **Attackers can spoil the training data.** This is called data poisoning.
- **Chatbots can be tricked** with prompt injection, where hidden text tells the AI to ignore its rules.
- **Employees may leak data** by pasting confidential information into public AI tools.

"To reduce these risks, I would limit access, check inputs, monitor the model, keep logs, and keep a human in the loop for important decisions."

# Part 6: A full answer you can speak (about 1 minute)

"AI plays a big role in cyber security because security teams get too much data for humans to read. AI helps in four main ways. First, it detects unusual behaviour, like a login from a strange place at a strange time. Second, it catches malware and phishing emails by studying their behaviour and content. Third, it helps the SOC team by sorting alerts and summarising incidents, so analysts can focus on real threats. Fourth, it helps banks and companies stop fraud in real time.

If I had to build something, I would start with a phishing detector. I would collect phishing and normal emails, create features like the text and the link details, train a model like Random Forest or BERT, and check precision and recall. Then I would connect it to the email system and retrain it regularly.

I also know that AI has risks. Attackers use AI for deepfakes and phishing, and AI models can be attacked by poisoned data or prompt injection. So I would always keep humans involved in important decisions."

---

# How a Machine Learning or Deep Learning Model Is Trained

**Running example:** a phishing email detector that decides if an email is "phishing" or "safe". I use the same example in every step so it is easy to follow.

## Overview: the 10 steps

1. Define the problem
2. Collect data
3. Clean data
4. Explore data (EDA)
5. Prepare data and create features
6. Split data into train, validation, and test
7. Choose a model
8. Train the model
9. Evaluate the model
10. Tune and improve, then deploy and monitor

## Step 1: Define the problem

Before touching any data, decide exactly what you want to solve.

- **What is the goal?** Find phishing emails.
- **What type of problem is it?** Classification, because the answer is a category (phishing or safe). If we predicted a number, like a house price, it would be regression.
- **How will we measure success?** For example, "catch at least 95 percent of phishing emails with very few false alarms."

A clear goal helps you choose the right data, model, and metric. Many projects fail because this step was skipped.

# Step 2: Collect data

A model learns only from data, so data quality decides model quality.

**Where does data come from?**

- Company databases and logs
- Public datasets (Kaggle, UCI, PhishTank)
- APIs and web scraping
- Sensors, apps, or user surveys

**For our example:** we collect 50,000 emails, where each is marked "phishing" or "safe". This marking is called a **label**.

**Things to check while collecting:**

- **Enough data:** deep learning needs far more data than classic ML.
- **Right data:** it should look like the real emails the model will see later.
- **Balanced data:** if 99 percent of emails are safe, the model may never learn phishing well.
- **Legal and privacy rules:** remove personal details, and follow rules like GDPR and India's DPDP Act.
- **Security:** attackers can add fake data to spoil a model (data poisoning), so trust only reliable sources.

# Step 3: Clean the data

Real data is messy. Cleaning means fixing problems so the model does not learn wrong things. A common saying is "garbage in, garbage out."

**Common cleaning tasks:**

| Problem | What we do | Example |
|---|---|---|
| Missing values | Remove the row, or fill with mean, median, or most common value | Sender name is blank, so we fill it with "unknown" |
| Duplicates | Delete repeated rows | The same email was saved twice |
| Wrong values | Correct or remove | Age = 250 |
| Outliers | Check them, then remove or keep | One email has 50,000 words |

| | | |
|---|---|---|
| Wrong data types | Convert them | A date saved as text |
| Inconsistent format | Make them uniform | "India", "IN", and "india" become "India" |
| Wrong labels | Re-check them | A safe email marked as phishing |

**For text data**, we also do this:

- Convert to lowercase
- Remove extra symbols and HTML tags
- Remove very common words like "the" and "is" (stop words)
- Reduce words to their base form ("running" becomes "run")

Remember that an outlier is not always an error. A huge data transfer in security logs may be a real attack, so look before deleting.

# Step 4: Explore the data (EDA)

EDA means Exploratory Data Analysis. Here we look at the data using numbers and charts to understand it.

**What we check:**

- How many rows and columns there are
- How many are phishing and how many safe (class balance)
- The average, minimum, and maximum of each column
- Which features are related to each other (correlation)
- Charts: histograms, box plots, and heatmaps

**Example findings:** "Phishing emails often have more links and words like 'urgent'." This tells us which features may be useful.

# Step 5: Prepare the data and create features

A model can only understand numbers, so we change the data into a numeric form.

**5.1 Encoding categories**
Change text categories into numbers.

- Label encoding: Low = 0, Medium = 1, High = 2
- One-hot encoding: "Red, Blue" becomes two columns with 0 or 1

**5.2 Turning text into numbers**

- **Bag of Words or TF-IDF:** count important words.
- **Embeddings (like Word2Vec or BERT):** turn each word or sentence into a list of numbers that carries meaning.

**5.3 Scaling numbers**

Features can have very different ranges, like age (0 to 100) and salary (0 to 1,00,000). Large numbers can dominate, so we scale them.

- **Normalisation:** squeeze values into 0 to 1.
- **Standardisation:** make the mean 0 and the spread 1.

Tree-based models like Random Forest usually do not need scaling.

**5.4 Feature engineering**

Create new useful columns from existing ones. For phishing emails:

- Number of links
- Length of the URL
- Does the link contain "@"?
- Age of the sender's domain
- Number of urgent words

**5.5 Feature selection**

Remove features that do not help. This makes training faster and reduces overfitting.

**5.6 Handling imbalanced data**

If phishing is only 2 percent of the data:

- Oversample the small class (for example SMOTE), or reduce the big class
- Use class weights so mistakes on phishing cost more

# Step 6: Split the data

We must test the model on data it has never seen. Otherwise we cannot know if it truly learned.

| Set | Typical share | Purpose |
|---|---|---|
| **Training set** | 70% | The model learns from this |
| **Validation set** | 15% | We tune and compare models using this |

| | | |
|---|---|---|
| **Test set** | 15% | Final exam, used only once at the end |

**Very important rules:**

- **Split first, then scale.** If you scale before splitting, information from the test data leaks into training. This is called **data leakage**, and it gives fake high scores.
- **Never tune using the test set.**
- For time-based data like logs, split by time (train on the past, test on the future).
- For imbalanced data, use a stratified split, so each set has the same phishing percentage.

**Cross-validation:** When data is small, we split it into k parts (for example 5). We train 5 times, each time using a different part for testing, and then average the scores. This gives a more reliable result.

# Step 7: Choose a model

**Classic ML (small or table data):**

- Linear and Logistic Regression: simple baselines
- Decision Tree, Random Forest, XGBoost: very good for table data
- SVM, KNN, Naive Bayes: other common choices

**Deep Learning (large data, images, text, audio):**

- CNN for images
- RNN or LSTM for sequences
- Transformer (BERT, GPT) for text

**Good habit:** Always start with a simple baseline model first. If a simple model already works well, you may not need a complex one. Choose by the type of data, the amount of data, the need for explanation, speed, and cost.

# Step 8: Train the model

Training means the model repeatedly makes predictions, checks how wrong they are, and adjusts itself to be less wrong.

## 8.1 Training in classic ML

The algorithm studies the training data and finds the best rules or numbers.

- **Linear regression:** finds the line with the least total error.
- **Decision tree:** finds the best questions to split the data.

- **Random Forest:** builds many trees and combines their votes.

Usually you call a function like `model.fit(X_train, y_train)`.

## 8.2 Training in deep learning (the loop)

A neural network has **weights**, which are numbers that start random and are slowly improved. One training round works like this:

1. **Take a small batch** of data (for example 32 emails).
2. **Forward pass:** the data goes through the layers, and the network makes a prediction.
3. **Calculate the loss:** a loss function measures how wrong the prediction is. A big number means very wrong.
4. **Backward pass (backpropagation):** the network works out how much each weight caused the error.
5. **Update weights:** an optimizer (such as Adam) changes the weights slightly to reduce the error. The step size is the **learning rate**.
6. **Repeat** for all batches. One full pass over all data is one **epoch**.
7. **Run many epochs** until the loss stops improving.

**Simple comparison:** It is like learning to throw darts. You throw (forward pass), see how far you missed (loss), work out what to change (backpropagation), adjust your arm a little (update weights), and throw again.

## 8.3 Important training terms

| Term | Simple meaning |
|---|---|
| **Weights** | Numbers the model learns |
| **Loss** | How wrong the model is |
| **Learning rate** | Size of each correction step |
| **Batch size** | How many examples are used for each update |
| **Epoch** | One full pass through the training data |
| **Optimizer** | The method that updates weights (SGD, Adam) |
| **Hyperparameter** | A setting we choose before training |

## 8.4 Watch the training

During training, watch two curves: the training loss and the validation loss.

- Both go down: learning is going well.
- Training loss goes down but validation loss goes up: **overfitting**. Stop early.
- Both stay high: **underfitting**.

Use **early stopping**, which stops training when the validation loss stops improving. Also save checkpoints so you can return to the best version.

# Step 9: Evaluate the model

Now we test how good the model is on data it has not seen (the validation and then the test set).

## 9.1 Metrics for classification

First, the **confusion matrix** counts four things:

| | **Predicted phishing** | **Predicted safe** |
|---|---|---|
| **Actually phishing** | True Positive (TP) | False Negative (FN) |
| **Actually safe** | False Positive (FP) | True Negative (TN) |

- **Accuracy** = correct predictions out of all. It can mislead when data is imbalanced.
- **Precision** = TP / (TP + FP). "Of the emails I called phishing, how many really were?" It matters when false alarms are costly.
- **Recall** = TP / (TP + FN). "Of all real phishing emails, how many did I catch?" It matters when missing an attack is dangerous.
- **F1-score** = the balance of precision and recall.
- **ROC-AUC** = how well the model separates the classes. 1.0 is perfect and 0.5 is a guess.

**Example of why accuracy fails:** If only 1 in 100 emails is phishing, a model that always says "safe" is 99 percent accurate, but it catches zero attacks. In security, we usually care a lot about recall, but we also watch precision so the analysts do not drown in false alerts.

## 9.2 Metrics for regression (predicting numbers)

- **MAE:** average size of the error
- **MSE and RMSE:** punish big errors more
- **R-squared:** how much of the pattern the model explains

## 9.3 Check beyond numbers

- Look at the mistakes. What kind of emails does it get wrong?
- Test on different groups to check fairness.
- Test against attacks (adversarial examples) for security-sensitive models.
- Check speed and cost, not only accuracy.

# Step 10: Tune and improve (fine-tuning)

"Fine-tuning" is used with two meanings. Both are explained here.

## 10.1 Hyperparameter tuning

Hyperparameters are settings we choose before training. We try different values and keep the best.

**Examples of hyperparameters:**

- Learning rate, batch size, number of epochs
- Number of layers and neurons
- Dropout rate
- Number of trees and depth of trees (Random Forest)
- The K in KNN

**Ways to tune:**

- **Grid search:** try every combination from a list. It is thorough but slow.
- **Random search:** try random combinations. It is usually faster and works well.
- **Bayesian optimisation:** uses past results to pick the next best try.

We compare the options using the **validation set** or cross-validation, never the test set.

## 10.2 Fixing overfitting and underfitting

| Problem | Signs | Fixes |
|---|---|---|
| **Overfitting** | Great on training, poor on validation | More data, regularisation (L1 and L2), dropout, simpler model, early stopping, data augmentation |
| **Underfitting** | Poor on both | Bigger model, better features, train longer, reduce regularisation |

## 10.3 Other ways to improve

- Get more or better data, because it often helps more than changing models.
- Fix wrong labels.
- Add better features.

- Combine models (ensembles).
- Try a different algorithm.

### 10.4 Fine-tuning a pretrained model (transfer learning)

This is common in deep learning and with LLMs.

1. Take a model already trained on a huge dataset (like BERT or ResNet).
2. Replace the last layer with one for your task.
3. First, freeze the early layers and train only the new layer.
4. Then, unfreeze some layers and train with a **very small learning rate**, so you do not destroy what it already knows.

**Why do this?**

- It needs much less data.
- It saves time and money.
- It often gives better results than training from scratch.

**Example:** Take BERT, which already understands English, and fine-tune it using 5,000 phishing and safe emails. It then becomes a phishing detector.

**Fine-tuning vs RAG:** Fine-tuning changes the model's weights and suits style and special tasks. RAG gives the model fresh outside documents at answer time and suits changing facts.

## Step 11: Final test, deployment, and monitoring

**Final test:** Once you have chosen the best model, test it once on the untouched test set. This number is your honest estimate of real performance.

**Deployment:** Put the model where it can be used.

- As an API or service
- Inside an app or an email gateway
- On a device or in the cloud

**Monitoring after deployment:** A model does not stay good forever.

- **Data drift:** the new data looks different from the training data. For example, attackers start using new phishing tricks.
- **Concept drift:** the meaning of the pattern changes.
- Track accuracy, precision, recall, and speed over time.
- Retrain regularly with new data.
- Keep logs, version your models and data, and secure the pipeline.

**Security checks for the whole pipeline:** protect the training data from poisoning, control who can change the model, protect the model file from theft, and test against adversarial inputs.

## One-minute summary you can say in the interview

"First, I define the problem and the success metric. Then I collect data and clean it by fixing missing values, duplicates, and wrong labels. I explore the data, create useful features, encode and scale them, and then split the data into training, validation, and test sets, taking care to avoid data leakage.

Then I choose a model, starting with a simple baseline, and train it. In deep learning, training means repeating a loop of forward pass, loss calculation, backpropagation, and weight update over many epochs. I watch the training and validation loss to catch overfitting.

After that, I evaluate the model using the right metrics, such as precision, recall, F1, and AUC, instead of accuracy alone. Then I tune the hyperparameters, and if needed, fine-tune a pretrained model on my own data. Finally, I test once on the test set, deploy the model, and keep monitoring it for drift and retraining when needed."

## Quick questions an interviewer may ask next

1. **Why do we split data into three sets?** The training set teaches, the validation set tunes, and the test set gives an honest final score.
2. **What is data leakage?** When information from the test data or the future sneaks into training and makes the results look better than they really are.
3. **How do you know if a model is overfitting?** Training score is high but validation score is low.
4. **Why is accuracy not enough?** With imbalanced data, a model can have high accuracy and still miss every attack.
5. **What is the difference between a parameter and a hyperparameter?** Parameters (weights) are learned by the model, and hyperparameters are set by us before training.

I can add this to your PDF as an extra section, or I can ask you these questions one by one as a practice round. Which would you prefer?

---

# Pandas, NumPy, Matplotlib and Scikit-learn: 5 interview questions each

I give 5 questions and answers for each library, so 20 in total. Each answer is simple, with a small code example and a real-world use. Most examples use security logs, since your interview is for a cyber security role.

# PART 1: PANDAS

## Q1. What is Pandas, and what is the difference between a Series and a DataFrame?

**Answer:** Pandas is a Python library used to load, clean, and analyse table-like data. A Series is one column of data, and a DataFrame is a full table made of many Series.

- A **Series** is 1-dimensional, like one column in Excel.
- A **DataFrame** is 2-dimensional, with rows and columns, like a whole Excel sheet.
- Each row has an **index** (a label), and each column has a name.
- Pandas can read CSV, Excel, JSON, and SQL data.
- **Real-world example:** A security analyst loads a firewall log file into a DataFrame to study it.

```
import pandas as pd
df = pd.read_csv("firewall_logs.csv")   # DataFrame
df["source_ip"]                         # Series (one column)
df.head()                               # first 5 rows
df.info()                               # columns, types, missing values
```

## Q2. How do you handle missing values in Pandas?

**Answer:** First we find the missing values, then we either remove them or fill them with a sensible value, depending on how much data is missing and why.

- `df.isnull().sum()` shows how many values are missing in each column.
- `df.dropna()` removes rows that have missing values.
- `df.fillna(value)` fills missing values with a number or text.
- For numbers, fill with the **mean** or **median**. For categories, fill with the **most common value (mode)**.
- **Real-world example:** In login logs, a missing "country" is filled with "Unknown". The missing value itself may be a useful signal.

```
df.isnull().sum()
df["age"] = df["age"].fillna(df["age"].median())
```

```
df["country"] = df["country"].fillna("Unknown")
df = df.dropna(subset=["user_id"])   # drop rows only if user_id is missing
```

## Q3. What is the difference between `loc` and `iloc`? How do you filter rows?

**Answer:** `loc` selects data by **labels** (names), and `iloc` selects data by **position** (numbers). Filtering uses a True or False condition on a column.

- `df.loc[0:2, "user"]` uses labels, and the end label **is included**.
- `df.iloc[0:2, 1]` uses positions, and the end position is **not included**.
- Filter with a condition like `df[df["status"] == "failed"]`.
- Combine conditions with `&` (and) and `|` (or), and put each condition in brackets.
- **Real-world example:** Select all failed logins from outside India.

```
failed = df[df["status"] == "failed"]
suspicious = df[(df["status"] == "failed") & (df["country"] != "India")]
df.iloc[0:5, 0:3]   # first 5 rows, first 3 columns
```

## Q4. What is `groupby` in Pandas, and how is it used?

**Answer:** `groupby` splits the data into groups, applies a calculation like count or average to each group, and combines the results.

- It follows the idea of "split, apply, combine."
- Common functions are `count`, `sum`, `mean`, `max`, and `min`.
- `agg()` lets you apply many functions at once.
- `value_counts()` is a quick way to count how often each value appears.
- **Real-world example:** Count failed logins per IP address to find a brute-force attacker.

```
failed = df[df["status"] == "failed"]
failed.groupby("source_ip").size().sort_values(ascending=False).head(10)

df.groupby("country").agg({"bytes": "sum", "user": "nunique"})
df["status"].value_counts()
```

## Q5. What is the difference between `merge`, `join`, and `concat`?

**Answer:** `concat` stacks tables on top of each other or side by side, while `merge` and `join` combine tables using a common column, like a SQL join.

- `pd.concat([df1, df2])` stacks rows (for example, logs from two days).
- `pd.merge(df1, df2, on="ip")` joins by a matching column, like SQL.
- Join types are **inner** (only matches), **left** (all from the left table), **right**, and **outer** (everything).
- `join` is a shortcut that merges using the index.
- **Real-world example:** Join the log table with a threat intelligence list of bad IPs to mark which log entries came from known attackers.

```
all_logs = pd.concat([day1, day2], ignore_index=True)
result = pd.merge(logs, bad_ips, on="ip", how="left")
```

# PART 2: NUMPY

## Q1. What is NumPy, and why is it faster than a Python list?

**Answer:** NumPy is a library for fast mathematical work on arrays of numbers. It is faster than a normal list because it stores same-type data together in memory and does the work in optimised C code.

- A NumPy array holds only one data type, so it is stored compactly.
- Operations apply to the whole array at once, with no slow Python loop.
- It uses less memory than a list.
- Pandas, Scikit-learn, TensorFlow, and PyTorch are all built on top of NumPy.
- **Real-world example:** Image pixels and network packet sizes are stored as NumPy arrays for quick calculation.

```
import numpy as np
a = np.array([1, 2, 3, 4])
print(a * 2)         # [2 4 6 8]  (a list cannot do this)
print(a.mean())      # 2.5
```

## Q2. What are `shape`, `dtype`, and `reshape` in NumPy?

**Answer:** `shape` tells the size of the array in each dimension, `dtype` tells the data type, and `reshape` changes the shape without changing the data.

- `shape` of (3, 4) means 3 rows and 4 columns.
- `ndim` gives the number of dimensions.
- `dtype` can be `int64`, `float32`, and so on. Smaller types save memory.
- `reshape(-1, 1)` lets NumPy work out one dimension by itself.
- **Real-world example:** A 28 x 28 image is flattened into 784 numbers before being given to a simple neural network.

```
a = np.arange(12)          # [0 1 2 ... 11]
b = a.reshape(3, 4)        # 3 rows, 4 columns
print(b.shape, b.dtype)     # (3, 4) int64
c = b.reshape(-1, 1)       # shape (12, 1)
```

## Q3. What is broadcasting in NumPy?

**Answer:** Broadcasting is the rule that lets NumPy do maths on arrays of different shapes by automatically stretching the smaller one to match the bigger one.

- It avoids writing loops and saves memory.
- Adding a single number to an array adds it to every element.
- A row of 3 values can be added to every row of a table with 3 columns.
- Shapes must be compatible, or NumPy gives an error.
- **Real-world example:** Subtract each column's average from all rows to centre the data (done before ML training).

```
data = np.array([[1, 2, 3],
                 [4, 5, 6]])
print(data + 10)                    # adds 10 to every value
print(data - data.mean(axis=0))          # subtract each column's mean
```

## Q4. What is vectorisation, and why is it better than a `for` loop?

**Answer:** Vectorisation means doing an operation on a whole array at once instead of one element at a time with a loop. It is much faster and the code is shorter.

- Python loops are slow because Python checks types at each step.
- Vectorised code runs in compiled C code inside NumPy.
- It can be 10 to 100 times faster on large data.
- Use built-in functions like `np.sum`, `np.where`, and `np.dot`.
- **Real-world example:** Checking 10 million packet sizes to flag those above a limit takes seconds with NumPy, but much longer with a loop.

```
sizes = np.random.randint(0, 5000, 1_000_000)
flags = np.where(sizes > 4000, 1, 0)      # vectorised, no loop
total = sizes.sum()
```

## Q5. What are `axis`, and the difference between a copy and a view?

**Answer:** `axis` tells NumPy in which direction to calculate, with `axis=0` going down the columns and `axis=1` going across the rows. A view shares the same data as the original array, while a copy is separate.

- `axis=0` gives one result per **column**, and `axis=1` gives one result per **row**.
- Slicing usually creates a **view**, so changing it changes the original.
- Use `.copy()` to make an independent copy.
- This is a common source of bugs.
- **Real-world example:** Take the average of each feature column before training a model, and make a copy before modifying the data.

```
x = np.array([[1, 2], [3, 4]])
x.sum(axis=0)     # [4 6]  column totals
x.sum(axis=1)     # [3 7]  row totals

v = x[0]          # view
v[0] = 99         # changes x too!
c = x.copy()      # safe copy
```

# PART 3: MATPLOTLIB

## Q1. What is Matplotlib and what are its common chart types?

**Answer:** Matplotlib is the main Python library for drawing charts and graphs, which helps us understand data visually.

- **Line plot:** shows change over time.
- **Bar chart:** compares categories.
- **Scatter plot:** shows the relationship between two numbers.
- **Histogram:** shows how values are spread.
- **Box plot:** shows spread and outliers.
- **Real-world example:** A line plot of login attempts per hour shows a sudden spike during an attack.

```
import matplotlib.pyplot as plt
plt.plot(hours, logins)
plt.title("Login attempts per hour")
plt.xlabel("Hour")
plt.ylabel("Attempts")
plt.show()
```

## Q2. What is the difference between a Figure and Axes? What are the two ways to plot?

**Answer:** The Figure is the whole canvas (the page), and the Axes is the actual plot area drawn on it. We can plot quickly with `plt` functions, or use the object-oriented way for more control.

- One Figure can have many Axes (many charts).
- The **pyplot style** (`plt.plot`) is quick and good for simple charts.
- The **object-oriented style** (`fig, ax = plt.subplots()`) is better for complex or multiple charts.
- Always add a title, axis labels, and a legend so the chart is clear.
- **Real-world example:** In a security report, you build clean, labelled charts using the object-oriented style.

```
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(hours, logins, label="Failed logins", color="red")
ax.set_title("Failed logins by hour")
ax.set_xlabel("Hour")
ax.set_ylabel("Count")
ax.legend()
plt.show()
```

## Q3. How do you choose the right chart for your data?

**Answer:** Choose the chart based on what you want to show: trend, comparison, relationship, or distribution.

- **Trend over time:** line chart.
- **Compare categories:** bar chart.
- **Relationship between two numbers:** scatter plot.
- **How data is spread:** histogram or box plot.
- **Share of a whole (small number of parts):** pie chart. Avoid it when there are many parts.
- **Real-world example:** Use a bar chart for "attacks by type" and a scatter plot for "file size vs entropy" to separate malware from normal files.

```
plt.bar(["Phishing", "Malware", "DDoS"], [120, 45, 30])
plt.hist(packet_sizes, bins=30)
plt.scatter(file_size, entropy, c=labels)
```

## Q4. How do you draw many charts in one figure (subplots)?

**Answer:** Use `plt.subplots(rows, columns)` to create a grid of charts in one figure, so you can compare them side by side.

- `fig, axes = plt.subplots(1, 2)` gives 1 row and 2 charts.
- `axes[0]` and `axes[1]` are the individual charts.
- `plt.tight_layout()` stops the labels from overlapping.
- `sharey=True` uses the same y-axis for fair comparison.
- **Real-world example:** Show a "before attack" and "during attack" traffic chart next to each other.

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4), sharey=True)
axes[0].plot(normal_traffic);  axes[0].set_title("Normal")
axes[1].plot(attack_traffic);  axes[1].set_title("Under attack")
plt.tight_layout()
plt.show()
```

## Q5. How do you save a chart, and what is the difference between Matplotlib and Seaborn?

**Answer:** Use `plt.savefig()` to save a chart as an image. Matplotlib gives full control, while Seaborn is built on top of Matplotlib and makes good-looking statistical charts with less code.

- Save with `plt.savefig("chart.png", dpi=300)` **before** `plt.show()`.
- Seaborn has ready charts like heatmap, pairplot, and countplot.
- Seaborn works nicely with Pandas DataFrames.
- Both are often used together.
- **Real-world example:** Use a Seaborn heatmap of correlations to see which features are related, and save it for a report.

```
import seaborn as sns
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm")
plt.savefig("correlation.png", dpi=300, bbox_inches="tight")
plt.show()
```

# PART 4: SCIKIT-LEARN

## Q1. What is Scikit-learn and what is its basic workflow?

**Answer:** Scikit-learn is a Python library with ready-made machine learning algorithms and tools for preparing data and evaluating models. Almost every model follows the same simple steps.

- Step 1: create the model object.
- Step 2: `fit(X_train, y_train)` trains the model.
- Step 3: `predict(X_test)` makes predictions.
- Step 4: check the score with metrics.
- Tools that change data (like scalers) use `fit()` and `transform()`.
- **Real-world example:** Train a Random Forest to classify network traffic as normal or attack, in just a few lines.

```
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print(model.score(X_test, y_test))
```

## Q2. How does `train_test_split` work, and what are `stratify` and `random_state`?

**Answer:** `train_test_split` divides the data into a training part and a testing part. `stratify` keeps the same class percentage in both parts, and `random_state` makes the split the same every time.

- `test_size=0.2` keeps 20 percent of the data for testing.
- `stratify=y` is very important for imbalanced data, such as rare attacks.
- `random_state=42` gives repeatable results, which is good for reports and debugging.
- The test set must never be used while training or tuning.
- **Real-world example:** If only 2 percent of events are attacks, stratify makes sure both sets have about 2 percent attacks.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
```

## Q3. What is a Pipeline, and how does it prevent data leakage?

**Answer:** A Pipeline joins several steps, like scaling and the model, into one object. It makes sure each step learns only from the training data, so the test data does not leak in.

- Wrong way: scaling the whole dataset before splitting lets information from the test set leak in.
- With a pipeline, `fit` runs scaling and training only on the training data.
- At prediction time, the same steps are automatically applied to new data.
- It keeps code clean and easy to deploy.
- **Real-world example:** One pipeline file that scales network features and then classifies attacks can be saved and used in production.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ("scale", StandardScaler()),
    ("clf", LogisticRegression())
])
pipe.fit(X_train, y_train)       # scaler learns only from training data
pipe.predict(X_test)
```

## Q4. How do you evaluate a classification model in Scikit-learn?

**Answer:** We use metrics from `sklearn.metrics`, such as the confusion matrix, precision, recall, F1-score, and ROC-AUC, because accuracy alone can be misleading.

- `confusion_matrix` shows correct and wrong predictions in a table.
- `classification_report` prints precision, recall, and F1 for each class.
- `roc_auc_score` measures how well the model separates the classes.
- For rare attacks, focus on recall and precision.
- **Real-world example:** An intrusion detection model may have 98 percent accuracy but only 40 percent recall, which means it misses most attacks.

```
from sklearn.metrics import confusion_matrix, classification_report, roc_auc_score
print(confusion_matrix(y_test, pred))
print(classification_report(y_test, pred))
print(roc_auc_score(y_test, model.predict_proba(X_test)[:, 1]))
```

## Q5. What are cross-validation and GridSearchCV?

**Answer:** Cross-validation tests the model on several different splits to get a reliable score, and GridSearchCV tries many hyperparameter combinations automatically using cross-validation to find the best one.

- `cross_val_score(model, X, y, cv=5)` trains and tests 5 times.
- It gives a more trustworthy score than a single split, especially for small data.
- `GridSearchCV` tests every combination from a list of settings.
- `RandomizedSearchCV` tries random combinations and is faster.
- After tuning, test once on the untouched test set.
- **Real-world example:** Tune the number of trees and the depth of a Random Forest to get the best fraud detection score.

```
from sklearn.model_selection import cross_val_score, GridSearchCV

scores = cross_val_score(model, X_train, y_train, cv=5, scoring="f1")
print(scores.mean())

params = {"n_estimators": [100, 200], "max_depth": [5, 10, None]}
grid = GridSearchCV(RandomForestClassifier(random_state=42), params, cv=5, scoring="f1")
grid.fit(X_train, y_train)
print(grid.best_params_, grid.best_score_)
```

## Quick tips for the interview

- If they ask "Which libraries do you use?", say: "NumPy for fast number work, Pandas for cleaning and analysing data, Matplotlib and Seaborn for charts, and Scikit-learn for training and evaluating models."
- Be ready to write 3 to 5 lines of code from memory: `read_csv`, `groupby`, `train_test_split`, `fit`, and `predict`.
- Mention cyber security use cases, like counting failed logins with `groupby` or detecting anomalies with Isolation Forest.
- If you do not remember the exact syntax, explain the idea and say you would check the documentation. This is acceptable in real work.

I can add these 20 questions to your PDF in the same red and green style, or I can give you a short coding practice set with a sample log file to try in Python. Which would you like?

PROJECT